02

Data modelling

Schema decisions, and why they're the expensive ones to change

Most of a project can be rewritten at any point: interface, logic, styling. The schema — the set of tables your data lives in — is the exception, because once real rows accumulate, changing the shape means migrating them, and some changes lose data with no way back. An assistant proposes a schema in seconds, which makes it easy to accept one in seconds; this chapter is for recognizing the decision points in that proposal while they're still cheap to change.

PART

The model

Tables, rows, columns, keys

A **table** holds one kind of thing: users, events, orders. A **row** is one instance of that thing; a **column** is one attribute every instance has. The **primary key** is the column that identifies a row permanently — usually a generated id rather than anything meaningful, since names, emails, and titles all change. A **foreign key** is a column holding another row's id, which is how tables refer to each other.

Relationships come in two shapes. One-to-many: an event has many RSVPs, each RSVP points at one event, so the RSVP row carries the event's id. Many-to-many: users belong to many groups and groups have many users, which takes a third table — one row per membership — because neither side can hold a list of the other's ids in a single column well.

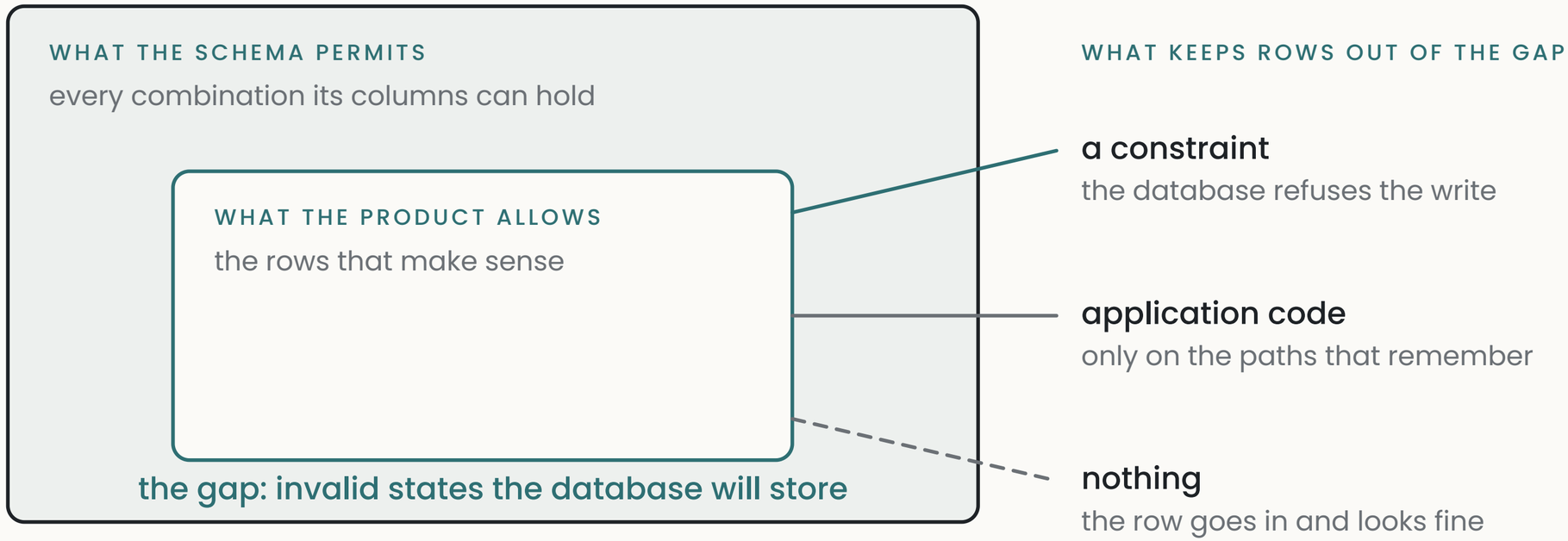
Nullability

A **nullable** column is one that may be empty. The design question is what empty means, and trouble starts when it means different things in different rows: an empty `ended_at` that means "still running" in one row and "we never recorded it" in another has put two facts in one column, and every query that touches it has to guess. Distinct meanings deserve distinct representation — a separate column, a status value, or a separate table.

Invalid states

A schema **permits** every combination of values its columns can hold; your product **forbids** some of them. An order that is both `cancelled` and `shipped_at` a date; an RSVP for an event with `capacity` already exceeded; a row with `discount_pct` of 340.

The gap between permitted and forbidden is where corrupt data comes from, and it has three possible guards: a **constraint** (a rule the database enforces on every write, like “this column is one of these four values”), application code (which guards only the paths that remember to), or nothing.



The cheapest wins are representational. A `status` column constrained to named values beats three booleans that can contradict each other; a rule written as a constraint holds against every writer, including code added later.

State machines

Anything with a `status` column is a state machine: a set of states plus the transitions you allow. `draft → published → archived` is three states and two legal moves; `archived → draft` either is legal or isn't, and the schema alone doesn't say. Writing the allowed transitions down — even as a comment — turns "the assistant wrote plausible status-handling code" into something checkable against a list.

Time

Store timestamps as absolute instants (UTC), and convert for display; a wall-clock time without a timezone is ambiguous the moment users or servers span two of them. Beyond that, three time patterns recur. `created_at` and `updated_at` on most tables, cheap and worth having from the start. **Soft delete** — a `deleted_at` column instead of removing the row — keeps history and enables undo, at a recurring cost: every read must now exclude deleted rows, and any query that forgets shows users data they deleted. **History tables** record every change rather than only the current value, which is the honest version of "we might need to know what happened."

What you hold about people

Some columns describe a person rather than a thing: an email, an address, a birthdate, free text they typed about themselves. Each is a liability as well as an asset — it can be leaked, it has to be deletable when the person leaves, and in most places keeping it needs a reason you could state. At recognition depth the rule is: hold what the product uses, know for how long, and have a path that removes it. Chapter 9 covers the same data on its way out to other services.

Why changing a schema is expensive

A **migration** is a script that changes the schema: add a table, add a column, change a type. Against an empty database it's trivial. Against real rows, three costs appear: existing rows need values for new columns (a **backfill**), some changes can't be undone (a dropped column's data is gone), and mistakes mid-migration can leave the data half-transformed. The practical consequence runs backwards: review effort belongs on schema proposals *before* rows accumulate, because that's the last point where changing your mind is free.

A JSON column against real columns

Most databases offer a JSON column: a place to store arbitrary structured data without declaring its shape. The trade is flexibility now against consequences later — no constraints, weaker querying, and every reader guessing at the contents' shape. It fits data that is genuinely irregular or that only gets stored and displayed. Anything you'll filter by, join on, or enforce rules about belongs in columns.

When tables get big

An **index** is a lookup structure the database maintains for a column, so a query filtering on that column stops scanning the whole table. Queries that are fast at a thousand rows and unusable at a million usually differ by one missing index. The related failure is the query with no upper bound: a list view that fetches every row works in the demo and grows with the table until it doesn't — lists need **pagination**, a limit plus a way to fetch the next batch.

PART

What goes wrong

There are ten entries: the first four concern the shape itself, the fifth what the schema holds about people, and the rest what the shape does to reads, writes, and growth.

1 A boolean where a table belongs

a flag standing in for something with variants.

ASK “For each boolean column, name the second and third variant a user might eventually want. Say none if there isn’t one.”

CHECK pick the most likely variant and ask what supporting it would require. If the answer is a migration plus a backfill, make the decision now rather than after the rows arrive.

TELL *columns like `repeat_weekly` or `is_premium`, where a second variant is plausible.*

2 Invalid states representable

the schema permits combinations the product forbids.

ASK “List the column combinations this schema permits that shouldn’t exist. For each, say what prevents it: a constraint, application code, or nothing.”

CHECK insert one of the forbidden combinations and see whether anything stops it.

TELL *nullable columns whose emptiness means different things in different rows, or booleans that can contradict each other.*

3 Current state stored alongside history

a value stored in two places with nothing keeping them equal.

ASK “Which columns can be derived from other tables? For each, say what keeps them in sync, and what happens if a write fails halfway.”

CHECK cause the drift deliberately in a test copy (chapter 0, D1), then see whether anything notices.

TELL *a `status` column plus an events or log table that also implies the status; a `total` column plus the line items it’s computed from.*

4 A destructive migration presented as additive

a change described as adding a feature that also loses data.

ASK “List every statement in this migration that loses data or breaks an existing read. For each, give the rollback.”

CHECK run it against a copy with real data before running it against the real thing.

TELL *a column rename, a type change, or a dropped default inside a migration whose description mentions none of them.*

5 Personal data with no reason and no exit

columns about a person that nothing needed and nothing deletes.

ASK “List every column that holds information about a person. For each: what the product does with it, how long it’s kept, and what happens to it when the person deletes their account. Write forever and nothing where those are the answers.”

CHECK delete a test account, then search every table for its rows.

TELL *emails, addresses, birthdates, or free text about users, kept with no stated purpose and no path that removes them when the account goes.*

6 Soft delete that still appears in reads

deleted rows shown because a query didn’t exclude them.

ASK “List every query that reads this table, and for each, whether it excludes soft-deleted rows. Say no explicitly.”

CHECK soft-delete a test row and look for it on every screen that lists the data.

TELL *a `deleted_at` column, and queries on the same table that never mention it.*

7 Timezone stored inconsistently

timestamps whose timezone depends on who wrote them.

ASK "For each timestamp column, state what timezone its values are stored in and what converts them for display. Say unknown where it's unknown."

CHECK write a row while pretending to be in another timezone, and compare what's stored with what's shown.

TELL *timestamp columns without timezone information, or values written by more than one path.*

8 Missing index on a filtered column

the query that slows as the table grows.

ASK "List every query that filters or sorts on an unindexed column, with the table's current row count and where it'll be in a year."

CHECK seed a copy with a realistic future row count and time the screen.

TELL *list views that filter or sort on a column no index covers.*

9 The unbounded query

a fetch whose size is the table's size.

ASK "List every query with no limit, and what its response becomes at ten times today's data."

CHECK the same seeded copy; load the screen and watch what arrives.

TELL *a list fetch with no limit.*

10 Orphaned files

the row deleted, the file it pointed at kept forever.

ASK "For each table that references stored files, state what happens to the file when the row is deleted. Say nothing where the answer is nothing."

CHECK delete a test row, then look for its file in storage.

TELL *a table referencing stored files, and deletion code that touches only the table.*

THE DIRECT TEST

Ask for the list of states your schema permits and your product forbids (the second entry's ask), pick the worst one, and insert it into a test copy (chapter 0's D1 prompt gets you one). What stops it — a constraint, an error from application code, or nothing — is the measure of how much your schema is enforcing on its own, and it's the guard that will still be there when new code writes to the same table.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Interrogate the schema

AUDIT

Here is my schema: [point your assistant at the project — on a hosted database the table editor holds it, and there may be a migrations folder]. Produce three lists. One: every boolean column, with the second and third variant a user might eventually want — ‘none’ where there isn’t one. Two: every nullable column, with each distinct thing empty can mean in it. Three: every column whose value could be computed from other tables, with what keeps it in sync. Full lists, as tables, no summarizing.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response has a row for every column that qualifies, and the second list is where findings usually live — any nullable column with two meanings listed is a real decision waiting. Follow up on each: “what would fixing this cost now, and at a hundred thousand rows?”

D2 · Explain my schema back to me

GROUNDED EXPLANATION

For each table in my schema, tell me: what real-world thing one row represents, what uniquely identifies it, which other tables it points to and why, and what one row’s lifecycle looks like from creation to deletion. Where a row’s meaning is ambiguous or a lifecycle has no ending, say so.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response reads like a description of your product. The findings are the places it doesn’t — a table whose row you can’t recognize, or a lifecycle that never ends, which is how tables grow forever.

D3 · Rehearse the migration

WALK THE FAILURE PATH

I’m considering this change: [the most likely future change, e.g. replacing a boolean with a set of variants]. Walk me through the migration with real data in place, step by step: what each step does to existing rows, what can fail at that step, and what state the data is in if we stop there. Then say what, if anything, is unrecoverable.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a numbered sequence where every step names a failure mode. If it presents the change as one step with no risks, ask specifically about rows written while the migration runs.

D4 · The invalid-state drill

PREDICTION QUIZ

Quiz me on my schema. One at a time, describe a row or combination of rows that my schema would accept, and ask me whether my product allows it. Wait for my answer, then tell me whether anything in the schema actually prevents it. Include at least one case involving two tables disagreeing and one involving a status that skipped a transition.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit before each answer. The cases where you say “the product forbids it” and the schema says nothing are the backlog of constraints worth adding.

D5 · Feel the index

BUILD A TOY

Help me seed a disposable copy of my database with a hundred thousand realistic rows in [the table that will grow most], then time my main list screen against it, then add the index you'd recommend and time it again.

WHAT A GOOD RESPONSE LOOKS LIKE

It takes about half an hour and converts "indexes matter" from a claim into two numbers from your own app. The seeded copy is also the fixture the entry checks above want.

WHERE THIS CONNECTS

Chapter 3 · Source of truth picks up what happens to this data at runtime, when the app holds copies of it. **Chapter 4 · The request lifecycle** covers the path a write takes to get here. **Chapter 9 · Third-party integrations** covers personal data leaving to services you don't control. The derived-columns entry and chapter 3's stored-against-derived section are the same idea on either side of the database boundary.